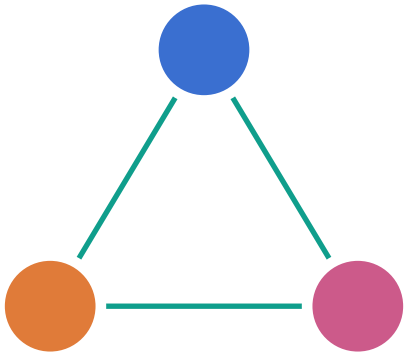
The Long-Distance Band

How we got friends in different houses
to play music together, in time



music travels straight between friends

A DISTRIBUTED SYSTEMS PROJECT, EXPLAINED SIMPLY

> **Everything in here is true and really works. We just took out the complicated words and put them at the back, so you can meet the ideas first.**

THE IDEA

What we made

Imagine you play the drums and your best friend plays guitar. You want to play a song together. But you are in your house, and they are in theirs.

You could try a video call. People do try. It sounds **awful**. You start together, then you drift apart, then you both slow down trying to fix it, and the song falls to pieces.

It is not because you are bad at music. It is because of something sneaky that the internet does, and almost nobody notices it.

> **We built a program that fixes it. This is how.**

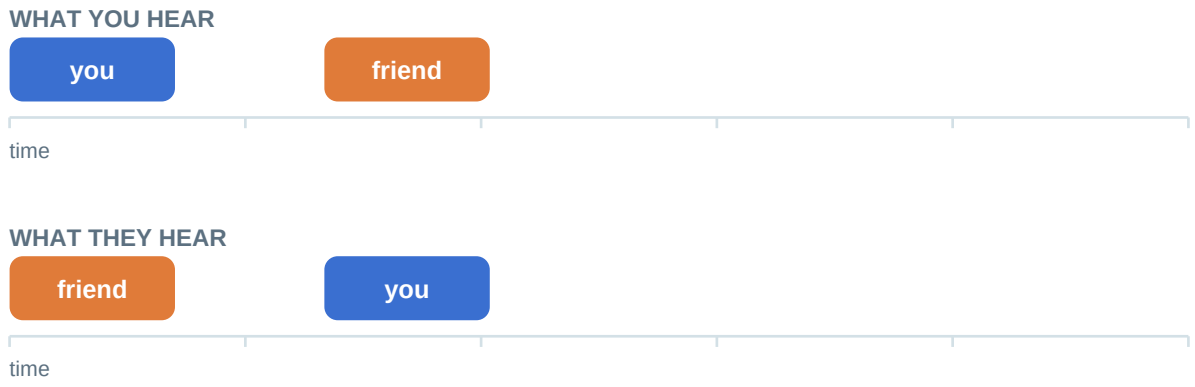
THE PROBLEM

Why is this so hard?

When you hit a drum, you hear it **instantly**. It happened right next to your ears.

But that sound has to travel all the way to your friend. That takes a moment. Not long, but long enough to matter, because music needs everybody to land on the beat together.

So here is what actually happens:



You both think the OTHER one is late.

Look carefully. **You** hear yourself first and your friend late. Your **friend** hears themselves first and you late.

So you slow down to wait for them. And they slow down to wait for you. And you both keep waiting for each other, slower and slower, until the song is a puddle on the floor.

> **You are each hearing a *different* version of the same song.**

THE CLEVER BIT

The surprising fix

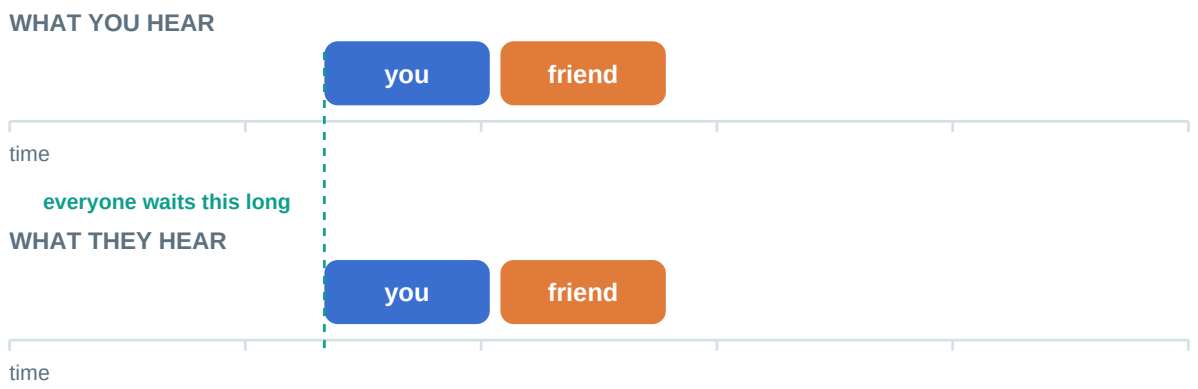
You would think the answer is "make the internet faster". It is not. We cannot make it faster, and anyway that is not really the problem.

Instead we did something that sounds silly at first:

> **We made everybody late. On purpose. By exactly the same amount.**

Think about a school assembly. If you sit right at the back, the teacher's voice reaches you a tiny bit late. Does that ruin assembly? Of course not. Everyone near you is hearing it late too. You are all late **together**, so it feels completely normal.

That is the whole secret. Being late does not ruin music. Being late by **different amounts** ruins music.



Both people hear exactly the same thing. It works.

So our program keeps checking how slow the slowest connection is, and then makes everyone wait exactly that long. Even you, listening to your own drums.

Yes, that means you hear your own drums slightly late, which feels strange for about a minute. Then your brain stops noticing, and the band is together. That is the trade we chose.

HOW IT WORKS

The four helpers

Our program has four helpers, and each one has exactly one job.

1

The Receptionist

You walk up and ask one question: "Can I join the band?" The receptionist answers straight away: "Yes, you are player number 2, and here is who else is here." One question, one answer.

2

The Announcer

Stands on a chair and shouts to the whole room: "New speed, 140 beats a minute!" Everyone hears it at once, and nobody has to be asked. Best part: if you arrive late, the announcer immediately tells you what you missed, so you are never left standing there confused.

3

The Direct Line

Once two friends know about each other, they get their own private phone line, and the music goes straight down it. It does not stop anywhere on the way. Every extra stop would make the music later, and later is the enemy.

4

The Counting Rule

This is the cleverest helper of all, so it gets its own page. Turn over.

THE BEST BIT

The counting rule

Every band needs to agree on when the beats happen. Here are two ways to do that.

The slow way	One computer shouts "CLAP!... CLAP!... CLAP!" and everybody claps when they hear it. Problem: if one shout gets lost or turns up late, somebody claps in the wrong place. And if the shouter goes away, everything stops dead.
Our way	We send **one** message: "Start at exactly 3 o'clock, and clap every half second." Now everyone can work out every single clap by themselves, forever. Nobody ever needs telling again.

> **It is the difference between being fed one spoonful at a time, and being handed the recipe.**

But hang on. What if your clock and your friend's clock do not agree? Then "3 o'clock" means different things and the whole plan collapses.

So before anything else, every computer checks its clock against one main clock, the way you might set your watch by the big clock at the station. It even allows for how long the checking itself took.

WHEN THINGS GO WRONG

What if somebody disappears?

One computer is the **leader**. It decides when the counting starts and how fast it goes.

So what happens if the leader's laptop runs out of battery in the middle of the song?

> **Nothing. The music just carries on.**

This is why the counting rule matters so much. Everyone already has the recipe, so nobody needs the leader in order to keep playing. The leader could vanish and you would not hear a single hiccup.

Meanwhile, every computer quietly says "still here!" every couple of seconds, a bit like a register being taken over and over. If someone stops answering for about six seconds, everybody agrees they have gone.

Then whoever has been in the band longest becomes the new leader. There is no argument and no vote, because everyone works it out the same way and so everyone picks the same person.

> **In our demo you can close the leader's window on purpose. The song does not even flinch.**

WHO IS WHO

Three ways to say who you are

Think about school for a second.

Your name	"Sam." Friendly and easy to say. But there might be three Sams in your class, and you could change it.
Your roll number	"24." Boring, but there is only one of you, and it never ever changes.
Your address	Where you actually live, so people can reach you. And it changes if you move house.

Computers need all three as well, and people mix them up constantly.

Our program always uses the **roll number** and never the address. Here is why that matters: a laptop can unplug from wifi and plug into a cable, which changes its address completely. It is still the same laptop and the same musician. If we had used the address, we would have lost them.

> **Names are for humans. Roll numbers are for keeping track. Addresses are just where to knock.**

A PUZZLE

Did that happen because of you?

Two children in a playground both shout "GO!" at the same moment. Neither of them heard the other one first.

So who shouted first?

It is a trick question. There is no answer, and that is not because the question is hard. It is because it is not a real question. Neither shout caused the other. They just both happened.

> **Computers call this concurrent: two things that happened without either one knowing about the other.**

You cannot solve this with clocks, because the clocks disagree. So instead every computer keeps a little counter, and passes it along with each message. By comparing counters we can always tell the difference between "this happened **because of** that" and "these two just both happened".

The lovely part is that you can watch it change. When the connection is fast, one player really does hear the other before playing, so the notes are genuinely in order. Add a delay, and now neither one can possibly have heard the other, so the notes become concurrent.

> **The network decides which one it is. You can see it flip on the screen, live.**

THE DEMONSTRATION

What you will see

1. **Two players join**
   Both screens show both names appearing. They have found each other.

2. **Press start**
   Both computers begin counting the beat, perfectly together.

3. **Play together**
   It works. This is the boring bit, and boring is the goal.

4. **Break it on purpose**
   We add a delay to one connection and switch the clever trick off. The band falls apart, and you can hear it happen.

5. **Fix it**
   Switch the trick back on. Everything snaps back together.

6. **Change the speed**
   Change it on one laptop. The other one changes too, straight away.

7. **Kill the leader**
   Close the leader's window. The music keeps going, and a new leader takes over by itself.

8. **Look at the numbers**
   Every measurement is on screen the whole time: how slow each connection is, how long everyone is waiting, which beat we are on.

> **The best moment is number 4. Breaking something on purpose, in front of everyone, is the clearest way to prove the fix was doing something all along.**

FOR THE GROWN-UPS

The proper words

Everything in this booklet is real. Here are the words the textbook uses for each idea, so you can find them again.

In this booklet	The proper name
The Receptionist	Remote Procedure Call (gRPC)
The Announcer	Message-Oriented Communication (MQTT, publish/subscribe)
Telling latecomers what they missed	Retained messages
The Direct Line	Peer-to-peer communication (WebRTC)
Introducing two computers to each other	Signalling (offer, answer, ICE)
Music flowing continuously	Stream-oriented communication
The Counting Rule	Shared timeline from an anchor
Setting your watch by the station clock	Cristian's algorithm (clock synchronisation)
Making everyone equally late	Latency compensation
The leader	Timing master
"Still here!" every two seconds	Heartbeat and failure detection
Longest-serving player takes over	Deterministic succession
Name, roll number, address	Names, identifiers and addresses
"Both happened, neither caused the other"	Concurrent events, happened-before
The little counters on messages	Lamport and vector clocks
Helpers that carry messages for you	Middleware
Music and sound over a network	Distributed multimedia system

One last honest note: this does not work over any distance at all. If the connection is really bad, everyone has to wait so long that the music stops feeling like music. Our trick buys agreement, not speed.